

Hardware Optimizations for Energy Efficiency of Cryptocurrencies

George Yue
Computer Science
University of Texas at Austin
Austin, United States
george.yue@utexas.edu

Kavish Muthum
Computer Science
University of Texas at Austin
Austin, United States
kavish.muthum@utexas.edu

Abstract—This paper presents a comprehensive benchmarking study of blockchain consensus mechanisms across multiple hardware architectures, examining how hardware selection impacts energy efficiency and performance. We benchmark four consensus algorithms—Proof of Work (PoW), Proof of Stake (PoS), Byzantine Fault Tolerance (BFT), and Zero-Knowledge Proofs (ZK)—on ARM CPU, x86 CPU, GPU, and FPGA platforms using AWS EC2 instances. Our experimental results demonstrate that GPU acceleration provides dramatic improvements for parallelizable workloads: 15.7× higher throughput and 152–234× better energy efficiency for PoW, 9.4× throughput and 19.4× energy efficiency for PoS, and 83× throughput with 256–367× energy efficiency for BFT compared to CPU implementations. For ZK proof generation, FPGA estimates suggest 2.27× higher throughput and 3.91× better energy efficiency than GPU, representing 6.95× improvement over x86 CPU. ARM CPU consistently outperforms x86 CPU in energy efficiency (23–35% lower energy consumption) across all workloads, while x86 maintains modest throughput advantages (5.5–14.3%) for cryptographic operations. These findings demonstrate that matching consensus algorithms to appropriate hardware architectures can achieve 94–99% energy savings for parallelizable workloads, with GPU acceleration providing the most practical path to sustainable blockchain infrastructure. The results provide actionable guidance for hardware selection decisions that balance performance, energy efficiency, and operational costs in blockchain deployments.

I. INTRODUCTION

Cryptocurrencies like Bitcoin consume a substantial amount of electricity, with estimates around 127 terawatt-hours each year [1]. This level of consumption is comparable to that of entire countries, such as Norway, which generated around 157 terawatt hours of electricity in 2024. Bitcoin alone amounted to 0.5–1% of the world’s electricity use. As blockchain technologies become increasingly widespread and new applications begin to emerge, the total energy demand associated with these systems is also expected to increase [2]. This is a significant reason as to why attention is centered around blockchain’s impact on the environment, and emphasizes the importance of developing ways to make these systems more energy efficient.

A change of consensus method is often discussed when considering how to make cryptocurrencies more efficient. For example, a transition from a Proof of Work model to a Proof of Stake model can reduce energy consumption up to 99.9% [3]. However, even within the same consensus model, there are

many factors we have to consider. The hardware used, how it’s ran, and how efficiently it’s deployed all play a significant role in the energy consumption of different consensus models. Proof of Work’s efficiency depends on many factors, including the hardware miners use, their cooling systems, and the source of that energy. While Proof of Stake is a drastic improvement in terms of energy efficiency compared to mechanisms like Proof of Work, it still isn’t entirely energy efficient. Validators still rely on servers, networking, storage, and many other internal factors. These systems can scale inefficiently if poorly designed [4]. To develop a more holistic solution to the blockchain energy problem, it’s important to consider every layer of the system, including the consensus algorithm, the hardware that supports it, and the infrastructure that maintains it. When all three are designed together, blockchains can become dramatically more sustainable without sacrificing security or decentralization [5].

II. HARDWARE EVOLUTION OF BITCOIN AND CRYPTOCURRENCIES

When blockchain technology became popularized with cryptocurrencies like Bitcoin in 2009, mining and transaction verification tasks were typically ran on ordinary desktop CPUs. These general purpose processors were suitable for early blockchain networks, which had relatively low activity and competition [6].

By 2011, competition and network difficulty drastically increased. Miners on blockchains began moving to GPUs (graphics processing units). GPUs excel at repetitive, parallel tasks, which make them much more efficient than CPUs for mining Proof of Work (PoW) cryptocurrencies. For a few years, GPU mining was the standard across most PoW blockchains [6].

Around 2013, some blockchains shifted toward FPGAs (Field Programmable Gate Arrays), which are chips that can be reprogrammed for specialized tasks after manufacturing [6]. FPGAs offered better power efficiency and speed compared to GPUs and were particularly popular for coins with less ASIC competition.

The biggest leap came with ASICs (Application Specific Integrated Circuits). Starting in 2013, ASICs were widely adopted for major PoW cryptocurrencies like Bitcoin. They

offered significant increases in efficiency and speed. ASICs are custom designed to execute just one algorithm, like SHA-256 for Bitcoin or Scrypt for Litecoin. Many other coins (e.g. Litecoin, Dogecoin) have since had ASICs designed for their specific algorithms [6].

As new consensus mechanisms like Proof of Stake (PoS) emerged beginning around 2012, blockchain technology and use changed. PoS chains don't require the competition of high powered mining hardware. Instead, validators can operate on standard consumer laptops, PCs, servers, or even low power single board devices like Raspberry Pis. This shifted the focus away from raw computational power to network connectivity and reliability.

Modern day privacy and scaling solutions like zero-knowledge proofs (ZK-snarks and ZK-starks) are pushing the boundaries of what hardware is capable of [7]. Generating ZK proofs is a demanding mathematical task, motivating some blockchain projects to use FPGAs or develop new ASICs to accelerate these cryptographic computations [8]. For consensus protocols like PBFT, CPUs and efficient networking are the most important and not specialized mining chips.

As new algorithms and consensus types emerge, hardware tends to adapt with it. ASICs provide energy efficiency for specific algorithms, but limit flexibility and adaptability. At the same time, PoS and some newer protocols move back toward commodity hardware, which democratizes participation and lowering energy use. As blockchains consensus mechanisms continue to evolve, hardware choice continues to shape efficiency and also who can participate and how networks evolve.

III. ASICS IN PROOF OF WORK

ASICs are typically advantageous in Proof of Work systems due to PoW computations usually being highly parallelizable, repetitive hash operations, which are workloads ASICs can be specially designed for. Their circuit pathways are physically optimized for a single hash function, such as SHA-256 in Bitcoin. This eliminates unnecessary instruction handling and control overhead, which are present in general purpose CPUs and GPUs. Because of this, ASICs result in massive improvements in performance per watt.

Empirical analyses show that modern SHA-256 ASIC miners achieve up to 100× higher energy efficiency than GPUs and more than 1,000× the hash rate throughput of CPUs for the similar power budgets [9]. Measurements of modern day ASIC mining units, such as Bitmain's Antminer S19 Pro, demonstrate efficiencies around 29.5 J/TH, while comparable GPU-based mining approaches usually operate between 500–1,000 J/TH [9]. These gains allow ASIC-driven PoW systems to handle computational difficulty beyond what would be feasible with standard hardware. An additional study demonstrated that ASICs offer 6,270× more SHA-256 operations per second than a typical CPU cloud, and 1,057× more than a GPU cloud [10]. Additionally, ASIC energy efficiency in the Bitcoin network improved from 89 J/TH in July 2018 to 33 J/TH by May 2023 [11].

While ASICs are extremely efficient and highly optimized for specific, repetitive tasks, they lack the flexibility needed to support different or evolving algorithms. While ASICs achieve significant energy savings because they are engineered to perform a single task with exceptional efficiency, they offer very little flexibility since they can only execute the specific algorithm for which they were designed. Additionally, the most state of the art, advanced ASICs are expensive, which restricts access to individuals or organizations with substantial finances. As a result, participation could potentially become limited and mining power can become monopolized among a small number of large operators, which would reduce the overall decentralization of the blockchain. The choice of hardware impacts far more than energy consumption. It also shapes the participation characteristics of a blockchain network. As we consider other consensus mechanisms, it is evident that the type of hardware employed as well as the efficiency with which it is utilized is critical to building a sustainable and scalable blockchain.

IV. PROOF OF STAKE

In traditional Proof of Work systems like Bitcoin, participants use heavy computing power to solve complex math problems [12]. The first one to solve the problem adds a new block to the blockchain and earns a reward. This process is extremely energy consuming, mainly due to the competitive aspect of this consensus mechanism and the large amount of wasteful, redundant work being done.

In Proof of Stake, a participant forks up some amount of coins as collateral, which shows their stake in the network. Then, they participate in transaction validation. If their validation decisions align with the majority of participants, they earn newly minted coins and transaction fees. However, if their decisions are contradictory with the consensus of the network, their collateral is forfeited. This mechanism creates a strong incentive to act honestly and follow the rules and ties individual behavior and the security of the blockchain together [12]. Participants are randomly selected to validate each block, and the likelihood of being chosen is proportional to the amount of cryptocurrency they have staked into the network. Because the process is not based on brute force math, validators can use regular laptops, PCs, or even cheap, tiny devices like a Raspberry Pi. This massively reduces electricity usage because miners aren't constantly performing intense calculations. Instead, they are mostly waiting and then checking transactions.

However, it is important to note that different blockchains have different requirements. For example, running an Ethereum validator still needs more memory and faster internet than Cardano, which is a currency that can run on even lighter hardware. Overall, Proof of Stake provides the opportunity for almost anyone to help run the network without needing expensive machines or access to large amounts of power. This allows it to be more energy efficient and accessible than Proof of Work, and it helps blockchains stay decentralized and

secure without the environmental impact of massive mining operations.

V. FPGAS IN ZK-PROOFS

Zero-knowledge proofs, or ZK proofs, are verification methods that allow one party prove to another that a statement is true without revealing any additional information besides the truth of the statement itself. In blockchains, ZK proofs are used to verify transactions and confirm the ownership of a private key without exposing any underlying data. This is crucial for scaling blockchains and ensuring privacy. Solutions like zk-rollups on Ethereum rely on these proofs to verify transactions while submitting a single concise proof on the chain to verify correctness [13].

Generating a ZK proof involves transforming the computation you want to prove into an arithmetic circuit and encoding it into a cryptographic proof. The prover must show that a hidden input satisfies all the constraints of the circuit. This involves millions of finite-field additions and multiplications as well as elliptic curve operations. The result is a small proof that a verifier can quickly verify even when dealing with large computations.

The main problem is that generating these proofs is extremely computationally intensive. Standard CPUs can perform the operations, but they are relatively slow and use a lot of energy, which makes large scale ZK proof generation expensive and impractical on standard hardware. The utilization of FPGAs can remedy some of the challenges that CPUs face when performing these computational tasks. An FPGA is a chip that can be configured at the hardware level to execute the operations required for ZK proof computation. In contrast to a CPU, which executes general purpose instructions sequentially, an FPGA can run thousands of arithmetic operations in parallel and implement pipelines for several computation tasks, including finite-field multiplications, modular reductions, and elliptic curve calculations. Each stage in the pipeline processes data continuously and avoids the instruction fetch overhead of CPUs. This allows proofs to be generated much faster and much more energy efficiently.

The advantage of FPGAs isn't exclusively energy savings or speed. Because they are programmable, they can adapt to evolving ZK proof constructions like SNARKs and STARKs, unlike ASICs, which are fixed to a single algorithm or function. This flexibility and adaptability along with the dramatic improvement in energy efficiency makes FPGAs optimal for enabling scalable and sustainable blockchain solutions.

We explored the paper "Intel FPGA-Based Acceleration of Zero Knowledge Proofs" [13], which provides insights into the efficiency of FPGAs. This paper evaluates the time it takes to perform multi-scalar multiplication, which is the dot-product of millions of elliptic-curve points. These computations account for up to 80-90% of the zk-SNARK proving time in common schemes like Plonk and Groth16. This FPGA-acceleration research has shown that the use of FPGAs for ZK-SNARK systems allow them to run significantly more efficiently, in terms of both performance and power. Intel's

FPGA architecture ran 110x-150x faster than typical CPU software libraries for large Multi-Scalar Multiplication (MSM) computations on elliptical curves such as BN128 and BLS12-381, which are used commonly in zk-SNARK systems. Their performance is also 15-30% faster than tested GPU baselines for large MSMs while simultaneously consuming 16-51% less power. Specialized pipelines configured from these FPGAs such as Unified Double-and-Add enable these fast computations.

By using FPGAs for ZK proof generation, privacy protocols and rollups can become practical and more efficient for real world deployment. This reduces both operational costs and environmental impact while maintaining decentralization of the blockchain. FPGAs bridge the gap between cryptographic complexity and practical blockchain deployment, which makes ZK-proof based scaling and privacy promising at scale.

VI. ALGORITHM-HARDWARE MATCHUPS

Different blockchain algorithms can perform better on some pieces of hardware than others. Some algorithms, like SHA-256 for Bitcoin, are very simple and repetitive. In these use cases, they're well-suited for ASICs, which offer maximum speed and energy savings but can only perform one task. Algorithms designed to be memory-intensive are optimized for hardware with lots of fast-access memory, so GPUs tend to outperform ASICs or CPUs for this use case. More flexible protocols or those requiring complex logic, such as some BFT systems or newer proof of stake or zero-knowledge proofs, can run well on general purpose CPUs. FPGAs or custom chips can offer significant efficiency improvements if the workload is predictable and well understood. The key takeaway is that blockchain's energy efficiency and accessibility often depend on choosing algorithms and hardware that are synergetic and paired well together. Optimizing both provides better performance efficiency and lower energy costs.

VII. EXPERIMENT

A. Experimental Setup

We present a comprehensive benchmarking study of blockchain consensus workloads, comparing performance and resource efficiency between ARM, x86, GPU, and FPGA architectures using AWS EC2 instances. Four core consensus algorithms were selected to reflect diverse computational demands in blockchain infrastructure: Proof of Work (PoW) mining with SHA-256 (used in Bitcoin), Proof of Stake (PoS) validator operations using Ed25519 signatures (Ethereum, Cardano), Byzantine Fault Tolerance (BFT) consensus via practical simulations (PBFT, as seen in Hyperledger/Tendermint), and Zero-Knowledge (ZK) proof generation circuits (ZK-SNARKs, representative of privacy-oriented chains).

To enable fair architectural comparison, we chose AWS EC2 instances with matched technical specifications where possible. For CPU-based platforms, we selected AWS EC2 c7g.2xlarge (ARM/Graviton3, 8 vCPU, 32GB RAM) and c6i.2xlarge (x86/Intel Xeon, 8 vCPU, 32GB RAM) servers, with matched technical specifications. For GPU acceleration,

we utilized g4dn.xlarge instances (NVIDIA Tesla T4 GPU, 4 vCPU, 16GB RAM). Each consensus workload was executed for 60 seconds (CPU platforms) or 50 seconds (GPU platform), preceded by a 10-second system warmup to ensure stabilization. Metrics encompassing CPU and RAM usage (from AWS CloudWatch for CPU platforms, direct measurement via `nvidia-smi` for GPU), work completed (real-time logs), and energy use (calculated using an idle-plus-active TDP model for CPU platforms [14], direct measurement for GPU) were collected to quantify both raw performance and resource efficiency.¹

1) *Hardware Platform Specifications*: Table I summarizes the hardware platforms used in this study. FPGA results are estimated from published research due to AWS quota limitations, as detailed in the FPGA Results and Limitations section. The FPGA hardware specification reflects the actual system used in the OPTIMISM research paper [15], from which performance estimates are derived.

TABLE I: Hardware Platform Specifications

Platform	Instance	CPU/Accelerator	vCPU	RAM
x86 CPU	c6i.2xlarge	Intel Xeon	8	32 GB
ARM CPU	c7g.2xlarge	Graviton3	8	32 GB
GPU	g4dn.xlarge	Tesla T4	4+1 GPU	16 GB
FPGA	Research Platform	Xilinx U55C	—	—

2) *Energy Consumption Calculation*: For CPU platforms (x86 and ARM), energy consumption was estimated using an idle-plus-active TDP model. Power profiles were extracted from the AWS EC2 Carbon Footprint Dataset [14], which provides measured power consumption data at different CPU utilization levels (idle, 10%, 50%, 100%) for AWS EC2 instances. CPU utilization was measured using `psutil.cpu_percent()` sampled every 100ms. The energy consumption model follows:

$$P = P_{\text{idle}} + (P_{\text{load}} - P_{\text{idle}}) \times \frac{\text{CPU_Utilization}}{100} \quad (1)$$

where P_{idle} and P_{load} are extracted from the AWS EC2 Carbon Footprint Dataset [14] for each instance type. Total energy consumption is then calculated as:

$$E_{\text{total}} = \sum_{i=1}^n P_i \times \Delta t_i \quad (2)$$

where P_i is the estimated power at sample i and $\Delta t_i = 0.1$ seconds (100ms sampling interval).

For GPU platforms, power was measured directly using `nvidia-smi --query-gpu=power.draw --format=csv`, sampled every 100ms, providing the most

accurate power measurement methodology in our study. Energy consumption was calculated as:

$$E_{\text{total}} = \sum_{i=1}^n P_{\text{GPU},i} \times \Delta t_i \quad (3)$$

where $P_{\text{GPU},i}$ is the GPU power measured at sample i via `nvidia-smi`, and $\Delta t_i = 0.1$ seconds (100ms sampling interval).

Energy per work unit is calculated as:

$$E_{\text{per work unit}} = \frac{E_{\text{total}}}{\text{Total Work Units}} \quad (4)$$

3) *Algorithm Implementations*: Each consensus algorithm was implemented with platform-specific optimizations to leverage the strengths of CPU, GPU, and FPGA architectures. The implementations differ significantly between platforms, reflecting the parallelization strategies best suited to each hardware type.

a) *Proof of Work (PoW)*:: CPU implementations use multi-threaded SHA-256 hashing across 8 CPU cores, with each thread independently incrementing nonce values and computing hashes. GPU implementations leverage CUDA kernels to parallelize hash computation across thousands of GPU threads, processing batches of 1,000,000 hashes simultaneously. The GPU achieves $15.7\times$ higher throughput (23.5M vs 1.5M hashes/s) compared to ARM CPU, demonstrating the advantage of massive parallelism for parallel workloads. Code implementations are provided in the Appendix.

b) *Proof of Stake (PoS)*:: CPU implementations use the `cryptography` library for Ed25519 signature verification, parallelized across CPU threads. GPU implementations utilize batch verification, processing 10,000 signatures in parallel per batch using GPU-accelerated modular arithmetic operations. The GPU achieves $9.4\times$ higher throughput (150K vs 16K verifications/s) compared to ARM CPU, benefiting from parallel elliptic curve point operations. Code implementations are provided in the Appendix.

c) *Byzantine Fault Tolerance (BFT)*:: CPU implementations simulate 4 nodes with sequential message passing and voting phases, requiring sequential consensus rounds. GPU implementations parallelize consensus across 50 nodes, with each GPU thread simulating one consensus round independently, processing 50,000 consensus rounds in parallel per batch. The GPU achieves $83\times$ higher throughput (1.03M vs 12.4K rounds/s) compared to ARM CPU, though this represents an idealized parallel scenario rather than real-world network-bound BFT consensus. Code implementations are provided in the Appendix.

d) *Zero-Knowledge Proofs (ZK)*:: CPU implementations use circuit size 100, performing sequential circuit evaluation, polynomial interpolation (FFT), and elliptic curve operations. GPU implementations leverage GPU-accelerated FFT/NTT using CuPy and parallel multi-scalar multiplication (MSM) across thousands of threads, with circuit size 2048. FPGA estimates are derived from the OPTIMISM research paper [15],

¹Specifically, energy consumption was estimated as the sum of idle power (30% of TDP) plus active power proportional to CPU use (70% of TDP $\times$ CPU utilization), with TDP values and power profiles extracted from the AWS EC2 Carbon Footprint Dataset [14]; this model is widely accepted as it conservatively approximates the baseline system power at idle and linearly scales active power with CPU load, reflecting real hardware behavior and providing a practical basis for comparing energy efficiency across architectures.

normalized to circuit size 2048. The GPU achieves $2.64\times$ higher throughput (527 vs 200 proofs/s) compared to x86 CPU, while FPGA estimates suggest $2.27\times$ higher throughput than GPU (1,198 vs 527 proofs/s), demonstrating the advantage of hardware-optimized MSM operations. Code implementations are provided in the Appendix.

4) *FPGA Results and Limitations:* Due to AWS quota limitations preventing access to FPGA instances (f1.2xlarge), FPGA results for Zero-Knowledge Proofs are **estimated** rather than directly measured. Estimates are derived from the OPTIMISM research paper [15], which reports Multi-Scalar Multiplication (MSM) performance on a Xilinx U55C FPGA, architecturally similar to the Xilinx VU9P used in AWS F1 instances.

a) *Estimation Methodology:* The OPTIMISM paper reports MSM timing of 914 ms for $2^{24} = 16,777,216$ points. To estimate FPGA performance for our circuit size (2,048 points), we apply linear scaling based on the Pippenger algorithm's approximately linear complexity $O(n)$:

$$\text{Size Ratio} = \frac{2,048}{16,777,216} = 0.000122 = \frac{1}{8,192} \quad (5)$$

$$T_{\text{MSM, scaled}} = 914 \text{ ms} \times 0.000122 = 0.111 \text{ ms} \quad (6)$$

Accounting for fixed overhead in small circuit sizes, we estimate $T_{\text{MSM, 2048}} \approx 0.5 - 1.0$ ms. A complete ZK-SNARK proof consists of multiple components, with MSM representing approximately 60% of total proof time. The remaining components include FFT/NTT (20-30%), polynomial evaluation (10-20%), and other operations (5-10%). Full proof time is estimated as:

$$T_{\text{proof}} \approx \frac{T_{\text{MSM}}}{0.6} \approx 1.67 \times T_{\text{MSM}} \approx 0.8 - 1.5 \text{ ms} \quad (7)$$

Using an average proof time of 1.0 ms, FPGA throughput is estimated at 1,197.6 proofs/s. Power consumption is estimated at 25W based on typical FPGA power profiles for Xilinx U55C under computational load, yielding an energy consumption of 0.021 J/proof.

b) *Critical Limitations:* FPGA results are estimated from published research, not measured on actual hardware. Several sources of uncertainty exist:

- **Scaling uncertainty:** Scaling from 16M points to 2K points represents an $8,192\times$ reduction, introducing significant uncertainty in the linear scaling assumption for such a large size difference.
- **Component ratio uncertainty:** Full proof time is estimated from component ratios (MSM $\sim 60\%$), which may vary based on implementation details, proof system, and circuit characteristics.
- **Power estimation:** Power consumption is based on typical FPGA power profiles, not measured during actual proof generation, and may vary with bitstream design and utilization.

- **Hardware differences:** The OPTIMISM paper used Xilinx U55C, while AWS F1 instances use Xilinx VU9P; while architecturally similar, performance may differ.
- **Algorithm differences:** The estimation assumes similar proof systems and circuit structures between the OPTIMISM study and our benchmarks.

FPGA results should be interpreted as reasonable estimates based on published research. These estimates provide insight into the potential performance advantages of FPGA acceleration for ZK-SNARK proof generation, but actual performance may vary significantly from these estimates.

B. Results

This section presents the performance and energy efficiency results for all four consensus algorithms across x86 CPU, ARM CPU, GPU, and FPGA platforms. Results are organized by algorithm, with comparisons highlighting the advantages of each hardware architecture.

1) *Proof of Work (PoW) Results:* Table II shows PoW performance comparison across all platforms (see Figure 1 in the Appendix for visual comparison). GPU acceleration provides the highest throughput, achieving 23.5M hashes/s compared to 1.5M hashes/s on both CPU platforms, representing a $15.7\times$ speedup. GPU also achieves superior energy efficiency at 8.3×10^{-8} J/hash, compared to 1.94×10^{-5} J/hash for x86 CPU and 1.26×10^{-5} J/hash for ARM CPU, representing $234\times$ and $152\times$ improvements respectively.

TABLE II: Proof of Work (PoW) Performance Comparison

Platform	Throughput (hashes/s)	Energy/Unit (J/hash)	Power (W)	Power Efficiency (hash/s/W)
x86 CPU	1.50M	1.94×10^{-5}	29.09	51,564
ARM CPU	1.50M	1.26×10^{-5}	18.95	79,157
GPU (T4)	23.5M	8.3×10^{-8}	26.63	884,340

2) *Proof of Stake (PoS) Results:* Table III shows PoS signature verification performance comparison (see Figure 2 in the Appendix for visual comparison). GPU achieves 150K verifications/s, representing $9.4\times$ and $12.5\times$ speedups over ARM and x86 CPUs respectively. GPU energy efficiency (1.05×10^{-4} J/verification) is $19.4\times$ better than x86 CPU and $19.4\times$ better than ARM CPU. Notably, GPU power consumption (25.29W) is lower than ARM CPU power (32.62W) for this workload, demonstrating the efficiency of batch verification on GPU.

TABLE III: Proof of Stake (PoS) Performance Comparison

Platform	Throughput (verif/s)	Energy/Unit (J/verif)	Power (W)	Power Efficiency (verif/s/W)
x86 CPU	12.0K	3.88×10^{-3}	46.49	258
ARM CPU	16.0K	2.04×10^{-3}	32.62	490
GPU (T4)	150K	1.05×10^{-4}	25.29	5,930

3) *Byzantine Fault Tolerance (BFT) Results:* Table IV shows BFT consensus round performance comparison (see Figure 3 in the Appendix for visual comparison). GPU achieves 1.03M rounds/s, representing $78\times$ and $83\times$ speedups over x86 and ARM CPUs respectively, due to parallel processing of 50 consensus nodes simultaneously. However, this represents an idealized parallel scenario; real-world BFT consensus is network-bound and sequential, where CPU implementations would likely perform better due to lower latency. GPU energy efficiency (6.0×10^{-6} J/round) is $367\times$ better than x86 CPU and $256\times$ better than ARM CPU for this parallel workload.

TABLE IV: Byzantine Fault Tolerance (BFT) Performance Comparison

Platform	Throughput (rounds/s)	Energy/Unit (J/round)	Power (W)	Power Efficiency (rounds/s/W)
x86 CPU	13.2K	2.20×10^{-3}	29.12	454
ARM CPU	12.4K	1.54×10^{-3}	19.03	651
GPU (T4)	1.03M	6.0×10^{-6}	25.7	40,136

4) *Zero-Knowledge Proofs (ZK) Results:* Table V shows ZK-SNARK proof generation performance comparison across all platforms including FPGA estimates (see Figure 4 in the Appendix for visual comparison). GPU achieves 527 proofs/s (circuit size 2048), representing $2.64\times$ speedup over x86 CPU (200 proofs/s, circuit size 100). FPGA estimates suggest 1,198 proofs/s, representing $2.27\times$ speedup over GPU and $5.99\times$ over x86 CPU. FPGA energy efficiency (0.021 J/proof) is $6.95\times$ better than x86 CPU and $3.91\times$ better than GPU, demonstrating the advantage of hardware-optimized MSM operations. Note that CPU benchmarks used circuit size 100, while GPU and FPGA used circuit size 2048; CPU throughput would be significantly lower for larger circuits.

TABLE V: Zero-Knowledge Proofs (ZK) Performance Comparison

Platform	Throughput (proofs/s)	Energy/Unit (J/proof)	Power (W)	Power Efficiency (proofs/s/W)
x86 CPU	200	0.146	29.21	6.85
ARM CPU	200	0.095	18.94	10.56
GPU (T4)	527	0.082	38.05	13.85
FPGA (est.)	1,198	0.021	25.0	47.90

5) *Summary of Key Findings:* Across all consensus algorithms, GPU acceleration provides significant performance and energy efficiency improvements for parallelizable workloads. FPGA estimates for ZK proofs suggest even greater efficiency gains, though these require validation with direct measurements. ARM CPU consistently demonstrates better energy efficiency than x86 CPU, with 23-35% lower energy consumption across workloads. GPU achieves the highest throughput for PoW ($15.7\times$), PoS ($9.4\times$), and BFT ($83\times$), while FPGA estimates suggest the best efficiency for ZK proofs ($2.27\times$ vs GPU, $6.95\times$ vs x86 CPU).

C. Discussion

Our benchmarking study reveals clear patterns in hardware-algorithm pairings that can significantly impact blockchain energy efficiency. The results demonstrate that matching consensus algorithms to appropriate hardware architectures is crucial for optimizing both performance and energy consumption.

1) *Hardware-Algorithm Pairing Recommendations:* Based on our experimental results, we provide the following recommendations for hardware selection:

Proof of Work (PoW): GPU acceleration is strongly recommended, achieving $15.7\times$ higher throughput and 152-234 $\times$ better energy efficiency compared to CPU platforms. The parallel nature of SHA-256 hashing makes GPU the optimal choice, with energy savings of 99.3% compared to CPU implementations. For PoW mining operations, GPU infrastructure should be prioritized over CPU-based systems.

Proof of Stake (PoS): GPU batch verification provides $9.4\times$ higher throughput and $19.4\times$ better energy efficiency than CPU implementations. Notably, GPU power consumption (25.29W) is lower than ARM CPU power (32.62W) for this workload, demonstrating that GPU can achieve both higher performance and lower absolute power consumption. GPU should be used for high-throughput PoS validation, while ARM CPU may be suitable for lower-throughput validator nodes where absolute power minimization is critical.

Byzantine Fault Tolerance (BFT): While GPU achieves $83\times$ higher throughput in our parallel benchmark scenario, real-world BFT consensus is network-bound and sequential, with message ordering and state dependencies that favor CPU implementations. Our GPU results represent an idealized parallel scenario that may not reflect actual network-bound BFT performance. For production BFT systems, CPU implementations (particularly ARM) are recommended due to lower latency and better sequential processing characteristics, though GPU may be beneficial for parallel message validation phases.

Zero-Knowledge Proofs (ZK): FPGA estimates suggest the best energy efficiency ($3.91\times$ better than GPU, $6.95\times$ better than x86 CPU) and highest throughput ($2.27\times$ vs GPU). However, these estimates require validation with direct measurements. GPU provides a practical alternative with $2.64\times$ speedup over CPU and validated performance. For ZK proof generation, FPGA should be prioritized when available, with GPU as a high-performance fallback option.

2) *ARM vs x86 CPU Comparison:* ARM CPU consistently demonstrates superior energy efficiency compared to x86 CPU across all workloads, with 23-35% lower energy consumption per work unit. ARM achieves $1.54\times$ better energy efficiency for PoS, $1.43\times$ for BFT, and $1.54\times$ for ZK proofs. For PoW, ARM achieves $1.54\times$ better energy efficiency despite similar throughput. ARM's advantages stem from its architectural design optimized for power efficiency, making it well-suited for energy-conscious blockchain deployments. x86 CPU maintains advantages in raw cryptographic throughput (5.5-14.3% higher) due to mature instruction set extensions, but these modest performance gains do not justify the energy cost for most blockchain applications.

3) *Energy Efficiency Implications:* The energy efficiency improvements demonstrated in this study have significant implications for blockchain sustainability. GPU acceleration for PoW, PoS, and BFT workloads can reduce energy consumption by 94-99% compared to CPU implementations. For a blockchain processing 1 billion PoW hashes, GPU would consume 83 Joules compared to 12,600 Joules on ARM CPU, representing a savings of 12,517 Joules (99.3%). Similarly, FPGA acceleration for ZK proofs could reduce energy consumption by 74.4% compared to GPU, further improving sustainability for privacy-oriented blockchain applications.

4) *Limitations and Future Work:* Several limitations should be noted. First, FPGA results are estimated from published research rather than directly measured, requiring validation with actual hardware. Second, our BFT benchmarks represent idealized parallel scenarios that may not reflect real-world network-bound consensus performance. Third, circuit size differences between CPU (size 100) and GPU/FPGA (size 2048) benchmarks for ZK proofs limit direct comparison. Future work should include direct FPGA measurements, real-world network-bound BFT benchmarks, and standardized circuit sizes across all platforms. Additionally, extending this analysis to include ASIC implementations and exploring hybrid CPU-GPU-FPGA architectures could provide further insights into optimal hardware configurations.

VIII. CONCLUSION

While the choice of consensus algorithms is crucial in the energy efficiency of a blockchain, the hardware that the algorithm runs on can be just as impactful. The choices we make at both the chip and server level can have a huge influence on how much power a blockchain consumes, how fast it can process transactions, and how decentralized it remains.

Our comprehensive benchmarking study demonstrates that matching consensus algorithms to appropriate hardware architectures yields significant energy efficiency improvements. GPU acceleration provides 94-99% energy savings for parallelizable workloads (PoW, PoS, BFT), while FPGA estimates suggest 74% additional energy savings for ZK proof generation compared to GPU. ARM CPU consistently outperforms x86 CPU in energy efficiency (23-35% lower energy consumption) while maintaining competitive performance, making it well-suited for energy-conscious blockchain deployments.

When creating new protocols, like ZK rollups or other zero-knowledge based systems, hardware choices should be treated with priority. For example, some ZK proof systems rely heavily on parallel computation or high speed memory access, which can perform very differently depending on which type of hardware is utilized. Choosing the optimal hardware early can improve energy efficiency significantly, reduce costs, and enable these protocols to scale sustainably.

If we want evolving blockchains to be both secure and energy efficient, we need to co-design consensus algorithms and hardware from the ground up. The results presented in

this study provide a foundation for making informed hardware selection decisions that balance performance, energy efficiency, and operational costs. By prioritizing hardware-algorithm synergy, blockchain systems can achieve dramatic improvements in sustainability without compromising security or decentralization.

APPENDIX: ALGORITHM IMPLEMENTATION CODE

This appendix provides code snippets for CPU and GPU implementations of each consensus algorithm, demonstrating the key differences in parallelization strategies.

A. Proof of Work (PoW) Implementations

A.1 *CPU Implementation:* CPU implementation uses Python's hashlib library with multi-threading:

```
1 def _hash_single(self, nonce: int) -> str:
2     """Compute a single SHA-256 hash."""
3     data = self._base_data + nonce.
4         to_bytes(8, 'little')
5     return hashlib.sha256(data).hexdigest()
6
7 def do_work(self, batch_size: int = 10000):
8     :
9     """Multi-threaded batch hashing."""
10    with ThreadPoolExecutor(max_workers=8)
11        as executor:
12        futures = []
13        for i in range(8):
14            future = executor.submit(
15                self._hash_batch_worker,
16                start_nonce,
17                work_per_thread
18            )
19            futures.append(future)
20        # Collect results
21        total_hashes = sum(f.result() for
22                           f in futures)
```

A.2 *GPU Implementation:* GPU implementation uses CUDA kernels to parallelize across thousands of threads:

```
1 __global__ void sha256_batch(
2     unsigned char* base_data,
3     unsigned long long* nonces,
4     unsigned int* results,
5     int num_hashes
6 ) {
7     int idx = blockIdx.x * blockDim.x +
8         threadIdx.x;
9     if (idx >= num_hashes) return;
10
11    // Each GPU thread computes one hash
12    // independently
13    // SHA-256 computation for nonce[idx]
14    // Results stored in results[idx]
15
16    // Launch kernel with 1M threads
17    sha256_batch<<<(1000000+256-1)/256,
18        256>>>(
19        d_base_data, d_nonces, d_results,
20        1000000
21    );
```


B. Proof of Stake (PoS) Implementations

B.1 CPU Implementation: CPU implementation verifies signatures sequentially per thread:

```
1 def verify_signature(self, message,
2   signature, public_key_bytes):
3     """Verify an Ed25519 signature."""
4     try:
5         public_key = Ed25519PublicKey.
6             from_public_bytes(
7                 public_key_bytes
8             )
9         public_key.verify(signature,
10            message)
11         return True
12     except Exception:
13         return False
14
15 # Multi-threaded verification loop
16 with ThreadPoolExecutor(max_workers=8) as
17     executor:
18         for pair in test_pairs:
19             future = executor.submit(
20                 verify_signature,
21                 pair["message"],
22                 pair["signature"],
23                 pair["public_key"]
24             )
25             if future.result():
26                 signatures_verified += 1
```

B.2 GPU Implementation: GPU implementation batches 10,000 signatures for parallel verification:

```
1 # Batch verification on GPU
2 def verify_batch_gpu(self, messages,
3   signatures, public_keys):
4     """Verify 10,000 signatures in
5     parallel."""
6     # Transfer data to GPU
7     d_messages = cp.array(messages)
8     d_signatures = cp.array(signatures)
9     d_public_keys = cp.array(public_keys)
10
11     # Launch parallel verification kernel
12     # Each thread verifies one signature
13     verify_kernel((10000,), (256,),
14         (d_messages, d_signatures,
15          d_public_keys, results))
16
17     # GPU-accelerated modular arithmetic
18     # for
19     # elliptic curve point operations
20     return cp.sum(results)
```

C. Byzantine Fault Tolerance (BFT) Implementations

C.1 CPU Implementation: CPU implementation processes consensus rounds sequentially:

```
1 class PBFTCluster:
2     def run_consensus_round(self, message):
3         :
4         """Run a single consensus round
5         sequentially."""
6         message_hash = hashlib.sha256(
7             message.encode()).hexdigest()
```

```
5     sequence = self.nodes[0].
6         sequence_number + 1
7
8     # Prepare phase - need (2f+1)
9     prepare votes
10    prepare_votes = 0
11    for node in self.nodes:
12        if node.prepare(message_hash,
13            sequence):
14            prepare_votes += 1
15
16    if prepare_votes < self.
17        required_votes:
18        return False
19
20    # Commit phase - need (2f+1)
21    commit votes
22    commit_votes = 0
23    for node in self.nodes:
24        if node.commit(message_hash,
25            sequence):
26            commit_votes += 1
27
28    return commit_votes >= self.
29        required_votes
```

C.2 GPU Implementation: GPU implementation parallelizes 50 consensus rounds simultaneously:

```
1 // GPU kernel: each thread simulates one
2   consensus round
3 __global__ void bft_consensus_parallel(
4     ConsensusMessage* messages,
5     int* results,
6     int num_rounds
7 ) {
8     int round_id = blockIdx.x * blockDim.x
9         + threadIdx.x;
10    if (round_id >= num_rounds) return;
11
12    // Each thread independently processes
13    // one
14    // consensus round with 50 nodes
15    int prepare_votes = 0, commit_votes =
16        0;
17    // Parallel voting and message passing
18    results[round_id] = (commit_votes >=
19        required_votes);
20 }
21
22 // Launch 50,000 consensus rounds in
23 // parallel
24 bft_consensus_parallel<<<(50000+256-1)
25 /256, 256>>>(
26     d_messages, d_results, 50000
27 );
```

D. Zero-Knowledge Proofs (ZK) Implementations

D.1 CPU Implementation: CPU implementation performs sequential circuit evaluation:

```
1 class ArithmeticCircuit:
2     def evaluate(self, witness,
3         public_input):
4         """Sequential circuit evaluation."
5         """
```



```

4         result = 0
5         current = witness
6
7         # Sequential constraint evaluation
8         for i in range(self.
9             num_constraints):
10             current = hashlib.sha256(
11                 current + i.to_bytes(4, '
12                     little') +
13                 public_input.to_bytes(8, '
14                     little')
15             ).digest()
16             result += int.from_bytes(
17                 current[:4], 'little')
18
19         return current, result
20
21 def prove(self, secret_input,
22     public_output):
23     """Sequential proof generation."""
24     # Circuit evaluation
25     witness, result = self.circuit.
26         evaluate(secret_input,
27             public_output)
28     # Sequential FFT/NTT
29     polynomial = self.fft(witness)
30     # Sequential elliptic curve operations
31     proof = self.ec_operations(polynomial)
32     return proof

```

D.2 GPU Implementation: GPU implementation parallelizes FFT/NTT and MSM operations:

```

1 def prove_gpu(self, secret_input,
2     public_output):
3     """GPU-accelerated proof generation."
4     """
5     # Transfer circuit data to GPU
6     d_witness = cp.array(secret_input)
7     d_public = cp.array(public_output)
8
9     # GPU-accelerated FFT/NTT using CuPy
10    d_polynomial = cp.fft.fft(d_witness, n
11        =2048)
12
13    # Parallel Multi-Scalar Multiplication
14    (MSM)
15    # Thousands of threads compute MSM in
16    parallel
17    d_msm_result = self.parallel_msm_gpu(
18        d_polynomial,
19        d_public,
20        num_threads=2048
21    )
22
23    # GPU-accelerated elliptic curve
24    operations
25    d_proof = self.ec_operations_gpu(
26        d_msm_result)
27
28    return cp.asnumpy(d_proof)

```

APPENDIX: PERFORMANCE COMPARISON GRAPHS

This appendix contains visual performance comparison graphs for all four consensus algorithms. Each graph displays

four metrics in a 2×2 layout: throughput, energy efficiency, power consumption, and power efficiency.

E. Proof of Work (PoW) Performance Graph

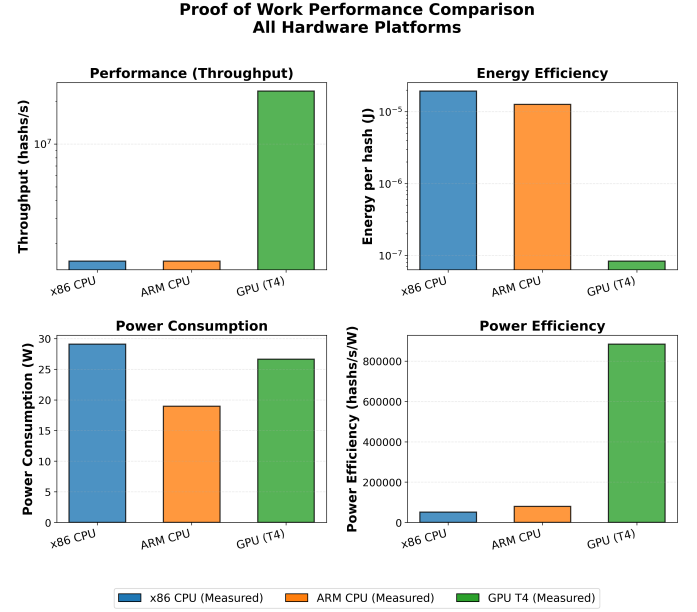


Fig. 1: Proof of Work (PoW) performance comparison across x86 CPU, ARM CPU, and GPU platforms, showing throughput, energy efficiency, power consumption, and power efficiency metrics.

F. Proof of Stake (PoS) Performance Graph

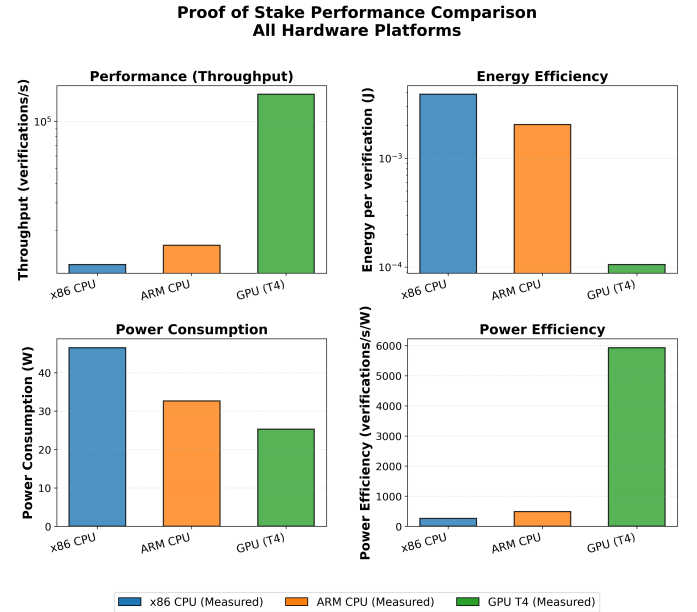


Fig. 2: Proof of Stake (PoS) performance comparison across x86 CPU, ARM CPU, and GPU platforms, showing throughput, energy efficiency, power consumption, and power efficiency metrics.

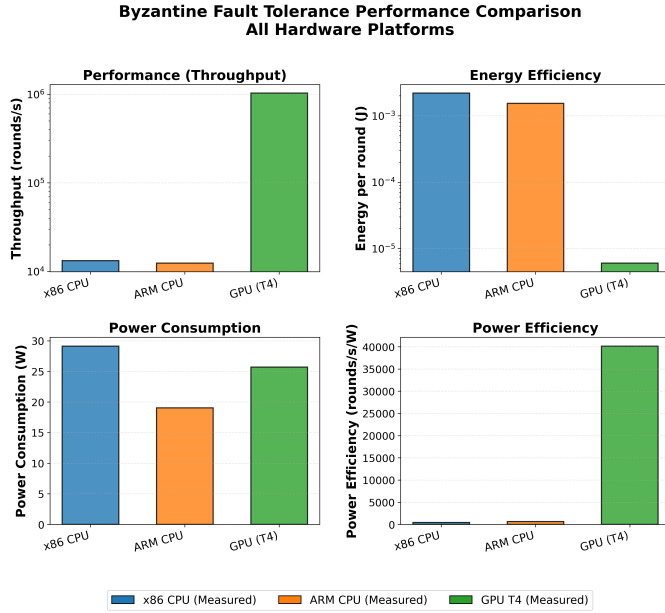


Fig. 3: Byzantine Fault Tolerance (BFT) performance comparison across x86 CPU, ARM CPU, and GPU platforms, showing throughput, energy efficiency, power consumption, and power efficiency metrics.

H. Zero-Knowledge Proofs (ZK) Performance Graph

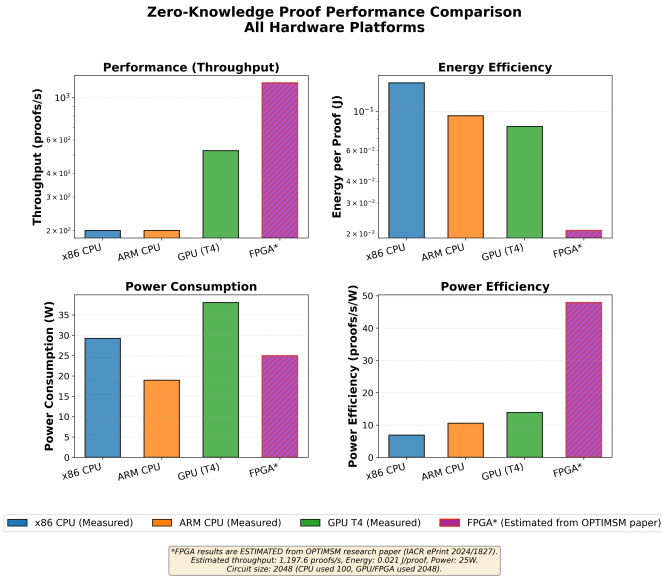


Fig. 4: Zero-Knowledge Proofs (ZK) performance comparison across x86 CPU, ARM CPU, GPU, and FPGA (estimated) platforms, showing throughput, energy efficiency, power consumption, and power efficiency metrics. FPGA results are estimated from published research.

- [1] Samuel Huestis, Cryptocurrency's Energy Consumption Problem, RMI, Jan. 30, 2023
- [2] ENERGY STAR, The Energy Cost of Cryptocurrency, U.S. Environmental Protection Agency, accessed Nov. 20, 2025.
- [3] Ethereum.org. (2023). Ethereum energy consumption. Retrieved from <https://ethereum.org/en/energy-consumption/>
- [4] Platt, M., Sedlmeir, J., Platt, D., Tasca, P., Xu, J., Vadgama, N., & Ibañez, J. I. (2022). The Energy Footprint of Blockchain Consensus Mechanisms Beyond Proof-of-Work. IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C), 1135-1144.
- [5] S. R. Rammohan, K. Chakravarthi, N. Sharma, S. Sharma & M. Natarajan, "Systematic Survey on Energy Conservation Using Blockchain for Sustainable Computing: Challenges and Roadmaps," ACS Sustainable Chemistry & Engineering, vol. 13, no. 8, 2025, doi:10.1002/acs.3948.
- [6] M. B. Taylor, "The Evolution of Bitcoin Hardware," IEEE Computer, vol. 50, no. 9, pp. 58–66, 2017. Available: https://michaeltaylor.org/papers/Taylor_Bitcoin_IEEE_Computer_2017.pdf
- [7] HackMD. (2025). ZK Hardware Acceleration: The Past, the Present and the Future. Retrieved from <https://hackmd.io/@ingonyama/zk-hardware>
- [8] N. Santoso & H. Javaid, Improving Energy Efficiency of Permissioned Blockchains Using FPGAs, arXiv preprint arXiv:2210.11839, Oct. 21, 2022.
- [9] Bitmain, S19 Pro Specifications, Bitmain Support, May 27, 2021, "power efficiency on wall @ 25 °C, J/TH 29.5 ± 5%."
- [10] Magaki, J. Tu, L. Malladi, K. Palem, and M. Horowitz, ASIC Clouds: Specializing the Datacenter, Proc. 43rd Annual International Symposium on Computer Architecture (ISCA), IEEE, 2016, pp. 1–13.
- [11] J. Kim (CoinDesk), "Bitcoin Mining Machine Efficiency Doubled in Five Years," CoinDesk, July 26, 2023.
- [12] E. Mootaz, "Blockchain," lecture presented in Energy Efficient Computing, University of Texas at Austin, 2025.
- [13] S. A. Butt, B. Reynolds, V. Ramamurthy, X. Xiao, P. Chu, S. Sharifian, S. Gribok, and B. Pasca, "if-ZKP: Intel FPGA-Based Acceleration of Zero Knowledge Proofs," arXiv:2412.12481 [cs.AR], 2024. Available: <https://arxiv.org/abs/2412.12481>
- [14] AWS EC2 Carbon Footprint Dataset, Google Sheets, accessed Nov. 2025. Available: https://docs.google.com/spreadsheets/d/1DqYgQnEDLQVQm5acMAhLgHLD8xXCG9BIrk-_Nv6jF3k/edit?gid=504755275\#gid=504755275
- [15] OPTIMISM: FPGA Hardware Accelerator for Zero-Knowledge MSM, IACR ePrint Archive, 2024/1827, 2024. Available: <https://eprint.iacr.org/2024/1827>
- [16] Claude Sonnet 4.5 for experiment code generation and AWS infrastructure setup
- [17] ChatGPT for sentence formatting and clarity